4a)Producer consumer problem using semaphores code:

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;           // counts empty slots
sem_t full;            // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

// Producer function
void *producer(void *arg) {
  int item, i = 0;

  while (1) {
    item = i++; // produce an item

    sem_wait(&empty);           // wait for empty slot
    pthread_mutex_lock(&mutex); // enter critical section

    buffer[in] = item;
    printf("Produced: %d at buffer[%d]\n", item, in);

    in = (in + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex); // leave critical section
    sem_post(&full);              // signal item available

    sleep(1); // simulate production time
  }
}

// Consumer function
void *consumer(void *arg) {
  int item;
```

```
  while (1) {
    sem_wait(&full);            // wait for filled slot
    pthread_mutex_lock(&mutex); // enter critical section

    item = buffer[out];
    printf("Consumed: %d from buffer[%d]\n", item, out);

    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex); // leave critical section
    sem_post(&empty);             // signal empty slot

    sleep(2); // simulate consumption time
  }
}

int main() {

  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  pthread_t prod, cons;

  sem_init(&empty, 0, BUFFER_SIZE);
  sem_init(&full, 0, 0);
  pthread_mutex_init(&mutex, NULL);

  pthread_create(&prod, NULL, producer, NULL);
  pthread_create(&cons, NULL, consumer, NULL);

  pthread_join(prod, NULL);
  pthread_join(cons, NULL);

  sem_destroy(&empty);
  sem_destroy(&full);
  pthread_mutex_destroy(&mutex);

  return 0;
}
```

Output:

Output

```
Name: Riya Gupta
USN:1WA24CS235

Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
```

4b) Dining-Philosopher's problem code:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void *philosopher(void *num) {
  int id = *(int *)num;
  int left = id;            // left fork index
  int right = (id + 1) % N; // right fork index

  while (1) {
    printf("Philosopher %d is thinking.\n", id);
    sleep(1); // Thinking

    // To avoid deadlock, philosophers with even id pick left then right,
    // odd id pick right then left.
    if (id % 2 == 0) {
      // Pick up left fork
      pthread_mutex_lock(&forks[left]);
      printf("Philosopher %d picked up left fork %d.\n", id, left);

      // Pick up right fork
      pthread_mutex_lock(&forks[right]);
      printf("Philosopher %d picked up right fork %d.\n", id, right);
    } else {
      // Pick up right fork
      pthread_mutex_lock(&forks[right]);
      printf("Philosopher %d picked up right fork %d.\n", id, right);

      // Pick up left fork
      pthread_mutex_lock(&forks[left]);
      printf("Philosopher %d picked up left fork %d.\n", id, left);
    }

    // Eating
```

```
    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    // Put down forks
    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
  }

  return NULL;
}

int main() {

  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int i;
  int ids[N];

  // Initialize forks (mutexes)
  for (i = 0; i < N; i++) {
    pthread_mutex_init(&forks[i], NULL);
    ids[i] = i;
  }

  // Create philosopher threads
  for (i = 0; i < N; i++) {
    pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
  }

  // Join threads (will never happen here, but good practice)
  for (i = 0; i < N; i++) {
    pthread_join(philosophers[i], NULL);
  }

  // Destroy mutexes (unreachable here)
  for (i = 0; i < N; i++) {
    pthread_mutex_destroy(&forks[i]);
  }

  return 0;
}
```

Output:

Output

```
Name: Riya Gupta
USN:1WA24CS235

Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
```